# 10. Design Collaborative Document Editing

**Key Terms:**

| Term | Full Name | What it does |
|---|---|---|
| **OT** | Operational Transformation | Algorithm transforming concurrent edits to maintain consistency |
| **CRDT** | Conflict-free Replicated Data Type | Data structure that auto-merges without conflicts |
| **WebSocket** | Full-duplex protocol | Real-time bidirectional communication |
| **Tombstone** | Deleted marker | Marks deleted items instead of removing (for sync) |
| **Pub/Sub** | Publish/Subscribe | Messaging pattern where senders broadcast to subscribers |

## Problem Statement

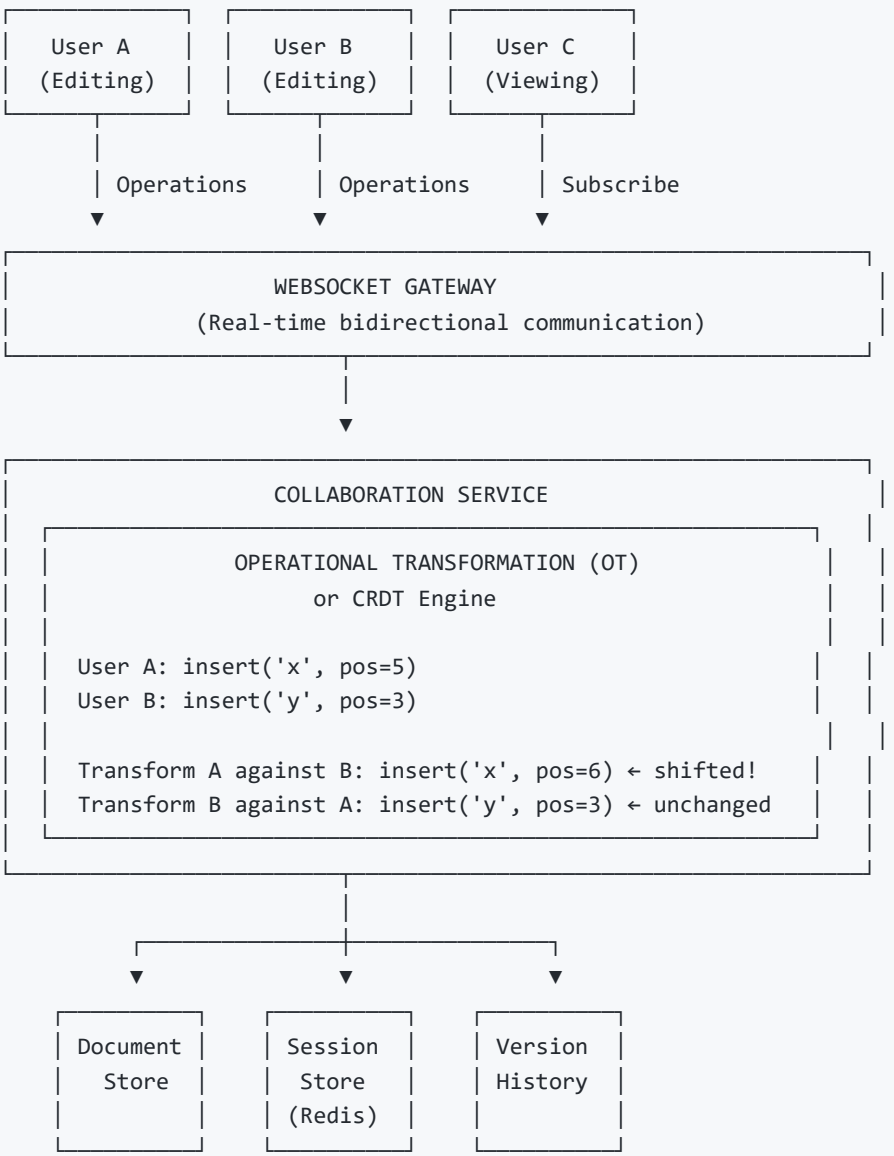
Design a real-time collaborative document editor like Google Docs or Microsoft Word Online where multiple users can edit simultaneously.

## Clarifying Questions to Ask

1. **Concurrency**: How many simultaneous editors? (50? 100?)
2. **Document size**: Max document size? (10MB?)
3. **Latency**: Max acceptable delay for seeing others' edits? (<100ms?)
4. **Offline support**: Allow offline editing and sync later?
5. **Features**: Comments, suggestions, version history?
6. **Conflict resolution**: How to handle simultaneous edits?
7. **Rich content**: Text only or images, tables, formatting?

## High-Level Design

```
┌─────────────────────────────────────────────────────────┐
│                COLLABORATIVE EDITING ARCHITECTURE       │
└─────────────────────────────────────────────────────────┘
```




# API Design

## Document APIs

| Method | Endpoint | Description |
|---|---|---|
| `POST` | `/api/v1/documents` | Create a new document |
| `GET` | `/api/v1/documents/{docId}` | Get document content and metadata |
| `GET` | `/api/v1/documents/{docId}/history` | Get revision history |
| `GET` | `/api/v1/documents/{docId}/history/{version}` | Get specific version |

| | | |
|---|---|---|
| `DELETE` | `/api/v1/documents/{docId}` | Delete a document |

### Collaboration APIs

| Method | Endpoint | Description |
|---|---|---|
| `POST` | `/api/v1/documents/{docId}/share` | Share document with users |
| `GET` | `/api/v1/documents/{docId}/collaborators` | List active collaborators |
| `GET` | `/api/v1/documents/{docId}/cursors` | Get collaborator cursor positions |

### WebSocket Events (Real-Time Editing)

| Event | Direction | Payload |
|---|---|---|
| `op.apply` | Client → Server | `{ docId, operation, baseVersion }` |
| `op.broadcast` | Server → Clients | `{ docId, operation, version, userId }` |
| `cursor.move` | Client → Server | `{ docId, position, selection }` |
| `cursor.update` | Server → Clients | `{ userId, position, selection, color }` |
| `user.joined` | Server → Clients | `{ userId, name }` |
| `user.left` | Server → Clients | `{ userId }` |

## Key Components

| Component | Purpose | Technology |
|---|---|---|
| WebSocket Gateway | Real-time sync | SignalR / Socket.io |
| OT/CRDT Engine | Conflict resolution | Custom implementation |
| Document Store | Persist documents | Cosmos DB / MongoDB |
| Session Store | Active editors | Redis |
| Version History | Track changes | Event store |
| Presence Service | Show active cursors | Redis pub/sub |

## Operational Transformation (OT)

```
PROBLEM: Concurrent edits can conflict

Initial: "HELLO"
User A: insert('X', 2) → "HEXLLO"
User B: delete(3)      → "HELO"

If both applied blindly:
  A then B: "HEXLLO" → delete(3) → "HEXLO" (wrong!)
  B then A: "HELO" → insert('X', 2) → "HEXLO" (wrong!)

OT SOLUTION: Transform operations

When A receives B's operation:
  transform(insert('X', 2), delete(3))
  → insert('X', 2) (position unaffected, deletion was after)

When B receives A's operation:
  transform(delete(3), insert('X', 2))
  → delete(4) (position shifted right by 1)

Result: Both converge to "HEXLO" correctly!
```

## CRDT Alternative

```
CRDT (Conflict-free Replicated Data Type):

Each character has a unique, ordered ID:
  "HELLO" → [(id1,'H'), (id2,'E'), (id3,'L'), (id4,'L'), (id5,'O')]

Insertions use IDs between existing characters:
  User A inserts 'X' after id2: (id2.5, 'X')
  User B deletes id3: mark (id3,'L') as tombstone

Merge is automatic - no transformation needed!
  Final: [(id1,'H'), (id2,'E'), (id2.5,'X'), (id4,'L'), (id5,'O')]
  = "HEXLO"

Pros: Simpler, works offline
Cons: Storage overhead (IDs), garbage collection
```

## Data Model

```
  // Document
  {
    "doc_id": "doc123",
    "title": "Project Plan",
    "content": [...],  // OT operations or CRDT state
    "version": 1542,
```

```
  "owner_id": "user456",
  "collaborators": ["user789", "user012"],
  "created_at": "2024-01-15T10:00:00Z",
  "updated_at": "2024-01-15T14:30:00Z"
}

// Operation (for OT)
{
  "op_id": "op789",
  "doc_id": "doc123",
  "user_id": "user456",
  "base_version": 1541,
  "operation": {
    "type": "insert",
    "position": 42,
    "text": "hello"
  },
  "timestamp": "2024-01-15T14:30:00Z"
}
```

## Scalability Considerations

- **Document sharding**: One collaboration session per server
- **Sticky sessions**: Route users of same doc to same server
- **Batching**: Batch operations (send every 50ms, not per keystroke)
- **Compression**: Delta compression for operation history
- **Snapshots**: Periodic snapshots to limit operation replay

## Trade-offs

| Decision | Option A | Option B | Recommendation |
|---|---|---|---|
| Algorithm | OT | CRDT | **CRDT** - simpler, offline-friendly |
| Sync | Per-keystroke | Batched | **Batched** - less overhead |
| Storage | Full doc | Operations | **Hybrid** - snapshots + ops |
| Cursors | Sync positions | Don't sync | **Sync** - better collaboration |

## Technologies Deep Dive

📚 **Learn more about the key technologies used in this design:**

| Technology | Role in This Design | Reference |
|---|---|---|

| | | |
|---|---|---|
| WebSocket | Real-time operation sync between clients | Tech Guide |
| Redis | Presence tracking and cursor pub/sub | Tech Guide |
| Raft | Consensus for operation ordering | Tech Guide |
| SignalR | Managed real-time connection service | Tech Guide |

---

## How to Explain It Well

> **Opening**: "Collaborative editing requires real-time sync with conflict resolution. I'll use CRDTs to ensure all users converge to the same state."

> **Key talking points**:
>
> - "CRDTs give each character a unique ID - merges are automatic"
> - "WebSocket for real-time, batch operations every 50ms"
> - "Presence shows other users' cursors using Redis pub/sub"
> - "Periodic snapshots avoid replaying entire operation history"
> - "Offline: queue operations locally, sync when reconnected"

---